39. Caching, Redis Patterns, Idempotency

39. Caching, Redis Patterns, Idempotency

Phase: Phase 6 — Systems and infrastructure
Estimated time: 1–2 days
Slug: `39-caching-redis`

Learning objectives

- Apply cache-aside pattern
- Choose TTLs and invalidation strategies
- Implement idempotency keys for writes
- Understand stampede risks

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Cache-aside

1. Read cache
2. On miss: read DB, populate cache
3. On write: update DB, invalidate key

Redis roles

- Cache
- Rate limiting
- Distributed locks (carefully)
- Queues / streams

Idempotency

For payment-like POSTs, clients send `Idempotency-Key`. Server stores result of first processing and replays it — critical for at-least-once delivery worlds.

Thundering herd

When hot key expires, many requests hit DB. Mitigate with singleflight / locking / soft expiry.

Common mistakes (especially from Python instincts)

- Caching without TTL
- Using Redis locks without expiry
- Assuming cache is always consistent with DB

Exercises

Work in `exercises/39-caching-redis/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/39-caching-redis/` until you have an honest failing attempt.

Self-check

1. What is cache-aside?
2. Why idempotency keys?
3. What is a stampede?

Further reading

- https://redis.io/docs/manual/patterns/